Codefest 4

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

Introduction

Deliverables are committed to your GitHub repository by the following Sunday at 11:59 pm. Each codefest is graded Satisfactory (S) / Not Yet (NY) as a CMAN+CLLM pair. A late submission counts as Not Yet and consumes one revision token. COPT is an optional problem that allows you to collect an extra token.

CMAN — Manual INT8 symmetric quantization

CMAN No AI No AI assistance permitted. Using a calculator or spreadsheet is fine.

You are given the following 4×4 FP32 weight matrix W:

```
W = [  0.85,  -1.20,   0.34,   2.10 ]
    [ -0.07,   0.91,  -1.88,   0.12 ]
    [  1.55,   0.03,  -0.44,  -2.31 ]
    [ -0.18,   1.03,   0.77,   0.55 ]
```

Tasks

1. **Scale factor.** Compute S using symmetric per-tensor quantization: S = max(|W|) / 127. Show the max value and the computed S.
2. **Quantize.** Quantize each element: W_q = round(W / S). Clamp to [−128, 127]. Write out the full 4×4 INT8 matrix.
3. **Dequantize.** Compute W_deq = W_q × S. Write out the 4×4 FP32 dequantized matrix.
4. **Error analysis.** Compute the per-element absolute error |W − W_deq|. Identify the element with the largest error and compute the *Mean Absolute Error* (MAE) across all 16 elements.
5. **Bad scale experiment.** Use S_bad = 0.01 (too small). Repeat quantization and dequantization. Compute the MAE. Explain in one sentence what goes wrong when S is too small.

CMAN deliverable

Commit `codefest/cf04/cman_quantization.md` (or .pdf / .png) containing:

(a) S computation with max value shown;
(b) 4×4 INT8 matrix W_q;
(c) 4×4 FP32 dequantized matrix W_deq;
(d) largest-error element identified, MAE computed;
(e) S_bad experiment with MAE and one-sentence explanation.

CLLM — LLM-assisted HDL: INT8 MAC unit generation and review

CLLM AI OK AI assistance is permitted and expected throughout this section.

Which LLMs to use

You must prompt **two different LLMs** and save both outputs. Record the exact model version you used (e.g., Claude Sonnet 4.6, GPT-4o) in your review document.

Prompting guidance

Structure your prompt with:

(1) the exact module signature including port names and bit widths;

(2) reset behavior (synchronous, active-high);
(3) what the output register should do each clock cycle;
(4) an explicit request for synthesizable SystemVerilog only — no initial blocks, no $display, no delays.

Example follow-up after the first result: *"Check your sensitivity list. Check for sign extension when multiplying 8-bit signed operands into a 32-bit accumulator. Confirm the module synthesizes with Yosys."*

MAC module specification
Prompt your chosen LLMs with the following specification. Keep it identical across both so the outputs are comparable.

Specification	**Module name:** mac **Inputs:** clk (1-bit), rst (1-bit, active-high synchronous reset), a (8-bit signed), b (8-bit signed) **Output:** out (32-bit signed accumulator) **Behavior:** On each rising clock edge: if rst is high, set out to 0; else add a×b to out. **Constraints:** Synthesizable SystemVerilog only. No initial blocks, no $display, no delays (#). Use always_ff.

Tasks
1. **Generate.** Prompt two different LLMs with the specification above. Save outputs as `codefest/cf04/hdl/mac_llm_A.v` and `codefest/cf04/hdl/mac_llm_B.v`. Record which LLM produced each file in `mac_code_review.md`.
2. **Compile.** Attempt to compile each file with `iverilog` or `verilator --lint-only`. Record any errors verbatim in `mac_code_review.md`.
3. **Simulate.** Write a testbench `codefest/cf04/hdl/mac_tb.v` or `mac_tb.py` (cocotb). Apply [a=3, b=4] for 3 cycles, assert rst, then apply [a=−5, b=2] for 2 cycles. Record all outputs in `mac_code_review.md`.
4. **Review.** In `codefest/cf04/review/mac_code_review.md`, document at least one (ideally 2-3) issue(s) across the two files. For each issue:
   - (a) quote the exact offending lines;
   - (b) explain why the code is wrong, ambiguous, or non-synthesizable;
   - (c) provide the corrected version.
5. **Correct.** Write `codefest/cf04/hdl/mac_correct.v` that compiles cleanly and passes the testbench. Run `yosys -p 'synth; stat' mac_correct.v` if available and include the output.

Common LLM failure modes to look for

Failure mode	What to look for
Non-synthesizable constructs	initial blocks, $display, delays (#5), fork/join, dynamic allocation
Wrong process type	always @(*) or plain always instead of always_ff / always_comb
Sign extension error	multiplying signed [7:0] x signed [7:0] without explicit cast; result treated as unsigned
Accumulator width mismatch	accumulating 16-bit product into 32-bit register without sign extension
Reset polarity error	active-low logic for an active-high spec, or vice versa
Missing port direction	ports declared without input/output keyword

CLLM deliverable
GitHub commit containing:
- `codefest/cf04/hdl/mac_llm_A.v` and `mac_llm_B.v` — one per LLM, model version identified in `mac_code_review.md`
- `codefest/cf04/hdl/mac_tb.v` or `mac_tb.py` — testbench
- `codefest/cf04/hdl/mac_correct.v` — compiles cleanly and passes testbench

- `codefest/cf04/review/mac_code_review.md` — at least 1 issue, with quoted lines, explanation, and correction
- Simulation log or screenshot showing `mac_correct.v` passing the testbench
- (Optional, no extra credit) Yosys synthesis output for `mac_correct.v`

COPT (optional) — cocotb testbench and your project compute core

COPT Optional	Completing this puts you on track for Milestone 2 (due Sun May 3).

Why this matters for M2: M2 requires a working HDL compute core with a testbench, an interface module with at least one verified transaction, and a documented precision choice. This challenge gets you started on all three.

Part A — Run a cocotb testbench on `mac_correct.v`
If you only wrote a Verilog testbench in CLLM, now rewrite it in cocotb using `codefest/cf04/cocotb_mac/` as your working directory. cocotb is the framework you will use throughout the project.

Minimal Makefile:
```
SIM        = icarus
TOPLEVEL_LANG = verilog
VERILOG_SOURCES = $(PWD)/mac_correct.v
TOPLEVEL   = mac
MODULE     = test_mac
include $(shell cocotb-config --makefiles)/Makefile.sim
```

Minimal test_mac.py:
```
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge

@cocotb.test()
async def test_mac_basic(dut):
    cocotb.start_soon(Clock(dut.clk, 10, units="ns").start())
    dut.rst.value = 1
    await RisingEdge(dut.clk)
    dut.rst.value = 0
    dut.a.value = 3; dut.b.value = 4
    for expected in [12, 24, 36]:
        await RisingEdge(dut.clk)
        assert dut.out.value.signed_integer == expected
    dut.rst.value = 1
    await RisingEdge(dut.clk)
    assert dut.out.value.signed_integer == 0
```

Part A tasks
1. **Run.** Execute `make SIM=icarus`. Commit the simulation log and VCD waveform file.
2. **Overflow test.** Add a second test function `test_mac_overflow` that checks behavior when the accumulator approaches $2^{31}-1$. Does the design saturate or wrap? Document the behavior.
3. **Waveform.** Open the VCD in GTKWave (`gtkwave dump.vcd`). Add clk, rst, a, b, and out. Screenshot to `codefest/cf04/cocotb_mac/waveform.png`.

Part B — First HDL module for your project compute core
1. **Write the module.** Implement the top-level module of your project's compute core in SystemVerilog. At minimum: correct port declarations, parameterized data width, and a reset-able register. Need not be fully functional yet.

2. **Write a testbench stub.** Create a cocotb testbench that instantiates your module, drives reset, and applies at least one representative input. Passing complex assertions is not required — the goal is a working simulation harness.
3. **Commit.** Commit both files to `project/hdl/`. Update your README: what the module does, which interface it will expose (SPI / AXI / other), and what precision you plan to use (INT8, FP16, etc.).
4. **Interface note.** Write one paragraph in your README justifying your interface choice based on your M1 arithmetic intensity. This is the bandwidth justification your M2 report will need.

COPT deliverables

- `codefest/cf04/cocotb_mac/test_mac.py` and Makefile — passing simulation
- Simulation log and GTKWave screenshot
- Overflow test with documented behavior (`test_mac_overflow`)
- `project/hdl/` — HDL module and testbench stub
- README update: module description, interface choice, precision justification